

defined yet (it is defined later, in a derived class). However, since we are invoking *bar()* from the base class constructor, it tries to call the pure virtual function; it is not possible to invoke a function that is not defined, and hence it results in a runtime exception (technically, it is 'undefined behaviour'). Note how we invoked *bar()* in the base class constructor—it is after casting the "this" pointer into the (*base **) type. If we attempt to directly call a pure virtual function, the compiler will give a compile-time error.

Now, let's look at a simple Java example. Can you predict its output?

```
class Base {
    public Base() {
        foo();
    }
    public void foo() {
        System.out.println("In Base's foo ");
    }
}

class Derived extends Base {
    public Derived() {
        i = new Integer(10);
    }
    public void foo() {
        System.out.println("In Derived's foo " +
i.toString() );
    }
    private Integer i;
}

class Test {
    public static void main(String [] s) {
        new Derived().foo();
    }
}
```

The program crashes with a *NullPointerException* ! Why? As I mentioned earlier, in Java, virtual functions are resolved to the dynamic type. Here, *foo* is a virtual function (in Java, all non-static, non-final methods are virtual) and we try invoking it from the constructor. Since it resolves to the dynamic type, the derived version of *foo* is called. Remember that we are still executing the base class constructor, and the derived constructor is yet to execute. Hence the private variable '*i*' inside *Derived* is not initialised yet (and all reference type variables are initialised to null in Java). Hence, the call *i.toString()* results in

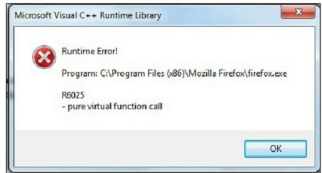


Figure 1: FirefoxPureVirtualCallError

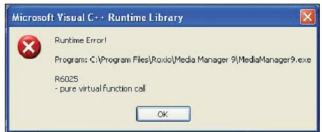


Figure 2: PureVirtualCall

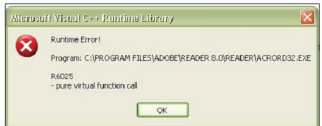


Figure 3: PureVirtualCallAcrobat

accessing the yet-to-be-initialised *Derived* object, and results in a *NullPointerException*.

The C# approach to calling virtual functions is similar to that of Java.

Calling virtual functions from constructors/ destructors is risky, no matter which OO language we use. Even if the program works in cases where virtual functions are called from constructors, the program can suddenly start failing if we extend the base classes in which such calls are present. Hence, it is a bad programming practice to call virtual functions from constructors/destructors, and most static analysers warn about this problem. 

By: S.G. Ganesh

S G Ganesh works for Siemens (Corporate Technology). You can reach him at sgganesh@gmail.com.